PHP end-of-term exam RETAKE (2022-23-1)

Due No Due Date **Points** None

Important information

- You have **180 minutes** to complete the exam. Filling in the included `README.eng.md` template, reading the tasks, downloading the materials, compressing and uploading your solution is **included** in the time limit.
- Solutions must be uploaded to ELTE Canvas. **The system closes at 19:00 (Hungarian time) and there will be no opportunity to submit your solution after the aforementioned time**.
- You may use **any kind of material** (including documentation, lectures, class exercises, cheat sheets, etc) during the exam, **except human help** (synchronous, asynchronous, chat, forums, etc)! By filling in the `README.eng.md` file, you accept the possible consequences of cheating.
- The tasks are independent and can be solved **in any order**.
- To begin your work you should first **download the starting package we have prepared**. Each tasks has its own directory in the package. All tasks are somewhat prepared. Other than the provided files, you are free to use any other files (to store data, organize your code, implement AJAX/fetch), but make sure your solutions are in the correct folder and can be started with the `index.php` file!
- The starting package contains a template called `README.eng.md` where you should fill in your name and Neptun code (the <> characters must be removed). **Submissions without a properly filled `README.eng.md` file will not be graded!**
- Each directory contains a file named `TASKS.md`. Replace the space in `[ ]` with an `x` where you have (even partially) succeeded to complete the task. This will make us easier to find your solution when we grade your exam.

Task 1: National Blue Trail tracks (1-tracks, 8 pts)

With high inflation such nowadays, we think twice about what we spend our money on. Expensive vacations can be replaced by excursions and the discovery of the natural wonders of Hungary. The National Blue Trail, which runs through the mountains from the western end of Hungary to the eastern end, is perfectly suitable for this. In order to select a section of this long trail, it would be good to know what parameters they have and how difficult they are. The `$tracks` array in the `index.php` file contains some of the most important data of all the 27 sections of the National Blue Trail (the number, the name of one and the other end of the section, the length of the track in km, the elevation in m, and the time in minutes). Display it as follows:

- a. (2 pts) Display the data as a table. Look for an example in `index.php`! The time must be shown in hours; steepness is the elevation per kilometer (level/distance); and the velocity is the distance traveled during the unit of time (distance/time, km/h)!
- b. (2 pts) Mark the length of the roads on a green-red color scale as the background colour of the distances! To do this, find the largest distance value, then for each section, calculate the percentage of the given section compared to the longest! (E.g. if the longest section is 72km, and the actual is 36km, then the percentage value is 50%.) This % value must be projected onto the 0-120 interval of the Hue value of the HSL color scale, so that 0% corresponds to the hue value 120, and 100% to the hue value 0. In CSS, an HSL value including transparency can be specified as follows: `hsla(0, 100%, 60%, 0.5)`. The current Hue value must be specified as the first parameter!

- c. (1 pt) Apply the same coloring to the elevation and steepness columns, as well!
- d. (1 pt) We also color the background color of the cells of the velocity column, but here in such a way that we first find the largest and smallest value, then the value of each section is given as a percentage compared to this interval (e.g. slowest 2km/h, fastest 8km/h, actual 5km/h is 50%), and this is colored so that the red (hue=0) should be at the smallest value, and green (hue=120) at the largest!
- e. (2 pts) Sum up the distance, elevation and time columns in the last row of the table!

Help: feel free to look around PHP's array functions to see if you can find efficient solutions, such as the `array_column` *(https://www.php.net/manual/en/function.array-column) function!*

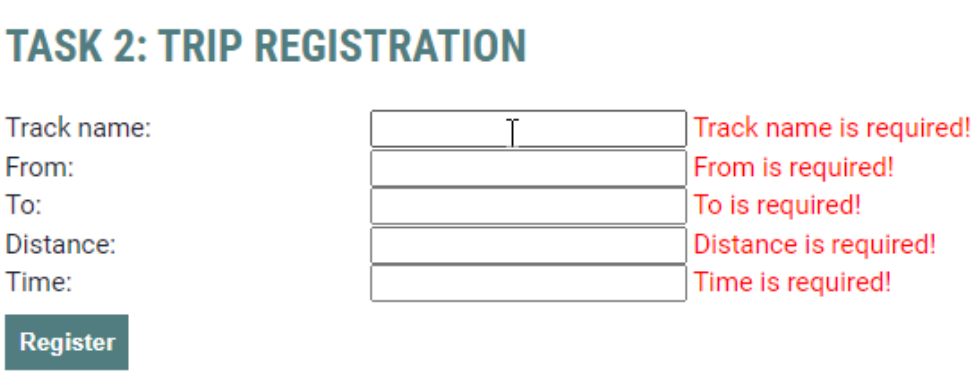
TASK 1: TRACKS

ID	FROM - TO	LENGTH (KM)	ELEVATION (M)	TIME (H)	STEEPNESS (M/KM)	VELOCITY (KM/H)
#1	Írottkő - Sárvár	70.2	430	18.3	6.1	3.8
#2	Sárvár - Sümeg	71.9	280	18.4	3.9	3.9
#3	Sümeg - Keszthely	47.5	810	13.2	17.1	3.6
#4	Keszthely - Tapolca	25.7	470	7.2	18.3	3.6
	•••					
#26	Boldogkőváralja - Sátoraljaújhely	52.7	1680	16	31.9	3.3
#27	Sátoraljaújhely - Hollóháza	46.4	1280	13.8	27.6	3.4
SUM		1099.1	26695	329.4		

Task 2: Trip registration (2-trip, 8 pts)

We would like to save the data of a new trip that we organise. The task is the **server-side** validation of an already structurally pre-created form (hence HTML level verification is not sufficient) according to the following conditions.

- a. (1 pt) The `trackname` field is required. The `distance` field is also required and can only be a number.
- b. (2 pts) The `from` and `to` fields are required and they must contain a settlement name that is included in the array `$places` from the starting file. The two fields cannot match!
- c. (2 pts) The `time` field is required and it must be in the format `X:XX` where the `X`s are digits. The highest allowed value before the colon (`:`) is 7, and the value after it must be less than 60.
- d. (1 pt) In case of incorrect input, an error message should appear next to the given field, informing about the cause of the error.
- e. (1 pt) The form placed on the page should keep its state which means that the submitted values should be written back into the fields.
- f. (1 pt) The `div` with ID `success` and its contents should only appear in the generated HTML code if the form was submitted by the user and no errors were found during validation.



Task 3: Logbook (3-logbook, 10 pts)

It is worth making small or large diary-like notes about our tours. In this task, we will create a PHP application in which we can make notes about the stages of the National Blue Trail, when, with whom, what experiences we gathered, and how we rate the tour!

The data must be stored in a file, the storage format is up to you. You may use the `Storage` class shown in the lectures but it's not mandatory. The track sections are contained in the `tracks.json` file. We can store our logs in another JSON file. *Note: the form does not need to be validated on server side in this task, we assume that it is filled in correctly!*

- a. (1 pt) As a first step, let us prepare to save new logs. There is a link on the `index.php` page that leads to `new.php`. Here fill in the drop-down field with the tour section data in the `tracks.json` file (in the format "Number. From - To")!
- b. (2 pts) In order to save a log on the `new.php` page, choose a trail section, enter the date range, write down your experiences, and finally specify the list of people we went with in the multi-line text field separated by commas (don't bother with the checkboxes for now!) and rate the tour! Save the data to a file by clicking the send button! One thing to note is that it is worth storing the hiking fellows separated by commas as an array! The `explode` function in PHP can be used for this! (Caution! It creates an array with one element for empty text!) After successful saving, direct the page to `index.php`!
- c. (1 pt) On the `new.php` page, generate checkboxes for the hiking fellows who were with us on our previous hikes. Make sure that each name only appears once!
- d. (1 pt) When saving on the `new.php` page, take into account that the hiking fellows can come from both the checkboxes or from the text input field!
- e. (1 pt) We list our tour descriptions on the `index.php` page, in such a way that the individual descriptions are listed section by section (see the sample given in `index.php`)!
- f. (1 pt) On the `index.php` page, do not list the sections that do not include a tour!
- g. (1 pt) On the `index.php` page, the tour descriptions should be links that can be clicked to get to the page displaying the details of the tour. Here, display the data according to the sample given on the `log.php` page. To display the description, it is worth using the `htmlspecialchars` and `nl2br` functions to get rid of sensitive HTML characters and display the multi-line text correctly!
- h. (1 pt) There should be a link on the `log.php` page that leads to a page (e.g. `edit.php`) where we can edit the tour description data! It is worth starting by creating a copy of the `new.php` page! On this page, as a first step, display the data of the given tour description!

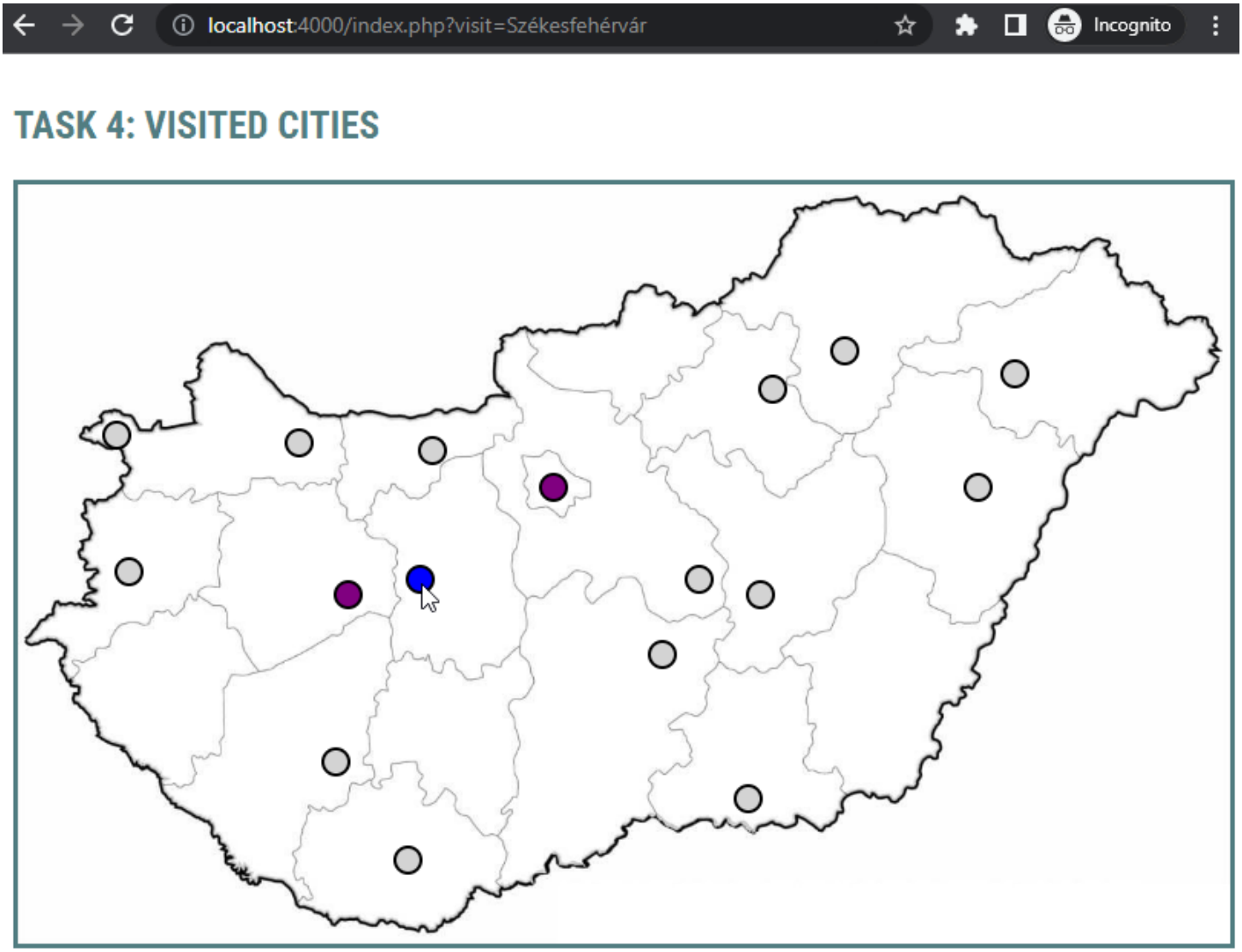
- i. (1 pt) When saving on the editing page, change the data of the given tour description and go back to the `log.php` page!



Task 4: Visited cities (4-visited, 6 pts)

During another tour, we have the opportunity to travel between certain settlements in several different orders. The `$places` array contains the names and coordinates of the individual settlements on a map (measured in % of height and width from the upper left corner), as well as the names of the settlements that can be reached from there by a safe road. Your task is to visualize a "graph traversal" with history stored in a **session** according to the points below, taking into account the locations and adjacency given in the starting file.

- a. (1 pt) Place all the settlements on the map given in the starting file using the examples that are already in the file.
- b. (1 pt) If there is no previous travel history in the session, then assume that we are currently in Budapest and have not visited any other city yet. Store this information **in a PHP session** in any format, and display the exact travelled path (currently Budapest only) under the map.
- c. (1 pt) Change the colour of the current city on the map to blue.
- d. (1 pt) Make those (and only those) cities that are reachable from the current location (see: `next` fields) purple and make their symbol on the map clickable. When the symbol is clicked, change our current location to that city. (This **should be done server-side** upon page reload, no client side solution or AJAX/fetch is needed in this task!)
- e. (1 pt) Change the colour of every city that we have visited before, but that does not belong to either of the two categories that have been coloured so far (current location or reachable from there) to green.
- f. (1 pt) The message at the bottom of the page should only appear if we have already visited every city at least once!


localhost:4000/index.php?visit=Székesfehérvár
Incognito
TASK 4: VISITED CITIES
Path: Budapest → Székesfehérvár